Object–oriented programming

Instructor: Masoud Asghari

Web page: http://www.masses.ir/lectures/oops2017sut

Ch: 6

In this slide

- We follow:
  - https://docs.oracle.com/javase/tutorial/index.html

Trail: Learning the Java Language > Packages

- how to bundle classes and interfaces into packages,
- how to use classes that are in packages,
- and how to arrange your file system so that the compiler can find your source files.

Creating and Using Packages

- You may bundle groups of related types into packages
  - **To make types easier to find and use**
    - You and other programmers can easily determine that these types are related.
  - **to avoid naming conflicts**
    - The names of your types won't conflict with the type names in other packages because the package creates a new namespace.
  - **to control access**
    - You can allow types within the package to have unrestricted access to one another yet still restrict access for types outside the package.
- A **package**
  - is a grouping of related types providing access protection and name space management.
  - Note that types refers to classes, interfaces, enumerations, and annotation types.
  - Examples: fundamental classes are in *java.lang*,
    - classes for reading and writing (input and output) are in *java.io*

Creating a Package

- To create a package
  - you **choose a name** for the package
  - and put a package statement with that name at the **top** of every **source file**
  - that contains the types (classes, interfaces, enumerations, and annotation types) that you want to include in the package.
- The package statement *package* name;
  - (for example, *package* graphics;)
  - must be the first line in the source file.
  - There can be only one package statement in each source file, and it applies to all types in the file.
- If you do not use a package statement
  - your type ends up in an **unnamed package**.
- Note: If you put multiple types in a single source file,
  - only one can be public, and it must have the same name as the source file.
  - For example, you can define public class Circle in the file Circle.java, define public interface Draggable in the file Draggable.java, define public enum Day in the file Day.java, and so forth.

Naming a Package

- Compiler allows both classes to have the same name if they are in different packages.
  - The fully qualified name of each Rectangle class includes the package name. That is, the fully qualified name of the *ExClass* class in the *exPackage* package is *exPackage.ExClass*
- **Naming Conventions**
  - Package names are written in all lower case to avoid conflict with the names of classes or interfaces.
  - Companies use their reversed Internet domain name to begin their package names
    - for example, *com.example.mypackage* for a package named *mypackage* created by a programmer at *example.com*.
  - Packages in the Java language itself begin with ***java.*** or ***javax.***

Using Package Members

- The types that comprise a package are known as the **package members**.
- To use a public package member from outside its package, you may do:
  - **Refer to the member by its fully qualified name**
    - You can use a package member's simple name if the code you are writing is in the same package as that member or if that member has been imported.
    - Otherwise you must use the member's **fully qualified name**, which includes the package name.
    - Example: *java.util.Scanner* myScanner = new *java.util.Scanner*(System.in);
  - **Import the package member**
    - To import a specific member into the current file, put an import statement at the beginning of the file before any type definitions but after the package statement
    - Now you can refer to the imported type (member) by its **simple name**.
    - Example: import java.util.Scanner;
      Scanner myScanner = new Scanner(System.in);
  - **Import the member's entire package**
    - To import all the types contained in a particular package,
    - use the import statement with the asterisk (*) wildcard character.
    - Now you can refer to any class or interface in the package by its **simple name**.
    - Example: import java.util.*;
      Scanner myScanner = new Scanner(System.in);

Using Package Members

- Java compiler
  - automatically imports two entire packages for each source file:
  - (1) the *java.lang* package and
  - (2) the current package (the package for the current file).
- **Name Ambiguities**
  - If a member in one package shares its name with a member in another package
  - and both packages are imported,
  - you must refer to each member by its qualified name.
- **The Static Import Statement**
  - The static import statement gives you a way to import the constants and static methods that you want to use
  - so that you do not need to prefix the name of their class.

Using Package Members

- **Example of Static Import Statement**
  - The *java.lang.Math* class defines the **PI constant** and many static methods, including methods for calculating sin, cos, tang, square roots, maxima, minima, exponents, ...
  - For example,

```
public static final double PI  = 3.141592653589793;
public static double cos(double a){
        ...
}
```

  - Ordinarily, to use these objects from another class, you prefix the class name

    For example `double r = Math.cos(Math.PI * theta);`
  - You can use the static import statement to import the static members of *java.lang.Math* so that you don't need to prefix the class name, *Math.*

  - The static members of Math can be imported either **individually**:

```
import static java.lang.Math.PI;
```

  - or as a **group**:

```
import static java.lang.Math.*;
```

  - Once they have been imported, the static members can be used without qualification.

    For example `double r = cos(PI * theta);`

Managing Source and Class Files

- Put the source code for
  - a class, interface, enumeration, or annotation type
  - in a text file whose name is the simple **name** of the type and whose **extension** is *.java*
  - For example:

```
//in the Rectangle.java file
package graphics;
public class Rectangle {
   ...
}
```

  - Then, put the source file in a directory whose name reflects the name of the package to which the type belongs:

```
.....\graphics\Rectangle.java
```

- When you compile a source file,
  - the compiler creates a different output file for each type defined in it.
  - The base name of the output file is the name of the type, and its **extension** is ***.class***

Managing Source and Class Files

- The full path to the classes directory,
  - <path_two>\classes,
  - is called the class path, and is set with the **CLASSPATH** system variable.
  - Both the compiler and the JVM construct the path to your .class files by adding the package name to the class path.
  - For example, if
    - <path_two>\classes
  - is your class path,
  - and the **package name** is
    - com.example.graphics,
  - then the compiler and JVM look for .class files in
    - <path_two>\classes\com\example\graphics.
- A class path may include several paths,
  - separated by a semicolon (Windows) or colon (UNIX).
  - By default, the compiler and the JVM search the current directory and the JAR file containing the Java platform classes so that these directories are automatically in your class path.